Notebook 1: Conventional Neural Network Training

This notebook illustrates the training of a very simple Neural Network to model a one dimensional data set. The data is fictitiously generated from solving

$$\frac{du}{dt} = -u^2$$

with initial condition $u(0) = 1.0$. Training data is generated for $t \in [0, 2]$. The intention is to try to use the Neural Network model to predict the solution for $t \in [0, 5]$. The Neural Network has one input, t and one output $u(t)$.

Loading libraries

In [40]:
```
using OrdinaryDiffEq
using CairoMakie
using OrdinaryDiffEqTsit5
using LinearAlgebra
using Lux
using Random
using Optimization
using OptimizationOptimisers
using ComponentArrays
using Zygote
```

Parameters of the training and validation data set

In [ ]:
```
#training parameters
max_iter=10_000
learning_rate=0.01

u0 = Float32[1.0]

# training data
datasize = 150
tspan = (0.0f0, 2.0f0)
#tspan = (0.0f0, 1.0f0)
tsteps = range(tspan[1], tspan[2]; length = datasize)

# validation data
tspan2 = (0.0f0, 5.0f0)
tsteps2 = range(tspan2[1], tspan2[2]; length = 200)

noise=0.002
#noise=0.01
random_initial_seed=3
```

3

Define governing equation

$$\frac{du}{dt} = -u^2$$

In [42]:
```
function trueODEfunc(du, u, p, t)
    du[1]=-u[1]*u[1]
end
```

```
trueODEfunc (generic function with 1 method)
```

Generating training data and add a bit of noise. The training data is computed for $t \in [0, 2]$

In [43]:
```
prob_trueode = ODEProblem(trueODEfunc, u0, tspan)
data = Array(solve(prob_trueode, Tsit5(); saveat = tsteps)) .+ noise*rand
```

```
1×150 Matrix{Float64}:
 1.006  0.995643  0.973659  0.963264  …  0.339126  0.335977  0.321598
```

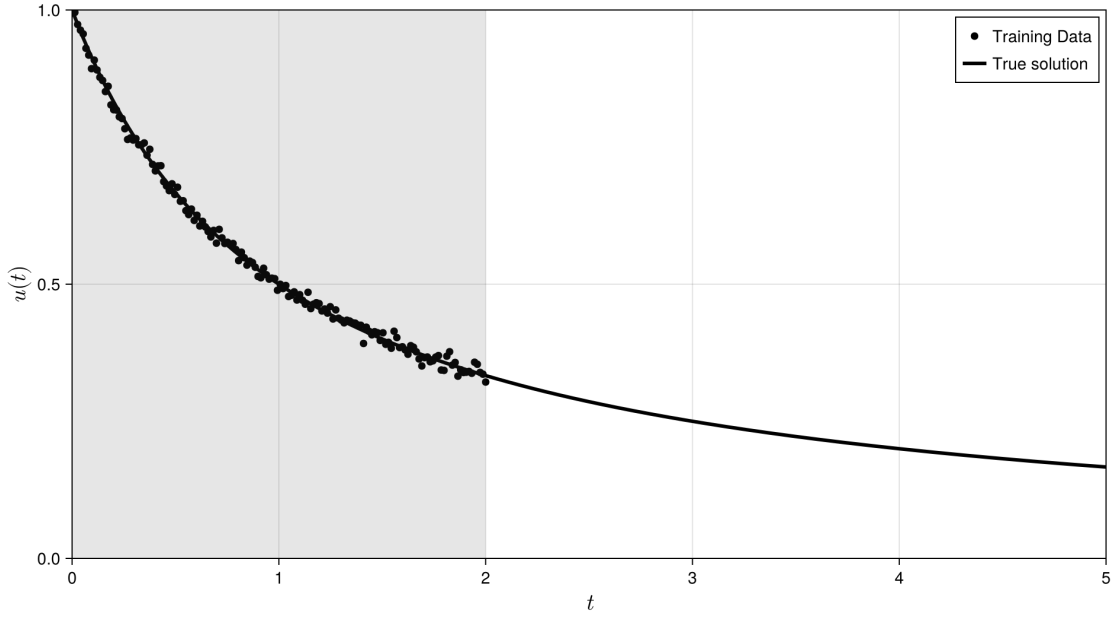
Generating the validation data, for $t \in [0, 5]$

In [44]:
```
#This is the prediction beyond the training data
prob_trueode_sol = ODEProblem(trueODEfunc, u0, tspan2)
data_true = Array(solve(prob_trueode_sol, Tsit5(); saveat = tsteps2))
```

```
1×200 Matrix{Float32}:
 1.0  0.97549  0.952153  0.929907  0.908676  …  0.168083  0.167376  0.1666
75
```

Plot the training and validation data to see how they look like.

In [45]:
```
fig = CairoMakie.Figure(size = (960, 540),backgroundcolor = :transparent)
ax = CairoMakie.Axis(fig[1, 1],
    backgroundcolor = :transparent,
    xlabelsize=20,
    ylabelsize=20,
    xlabel=L"t",ylabel=L"u(t)",title="")
CairoMakie.scatter!(ax,tsteps, data[:];color=:black,label="Training Data"
CairoMakie.lines!(ax,tsteps2, data_true[:]; linewidth = 3,color=:black,la
CairoMakie.vspan!(ax,tspan[1], tspan[2];color=(:grey, 0.1))
axislegend(ax;position=:rt)
xlims!(ax,tspan2[1],tspan2[2])
ylims!(ax,0,1)
#save("figures/01_TrainingRegion.png", fig)
fig
```



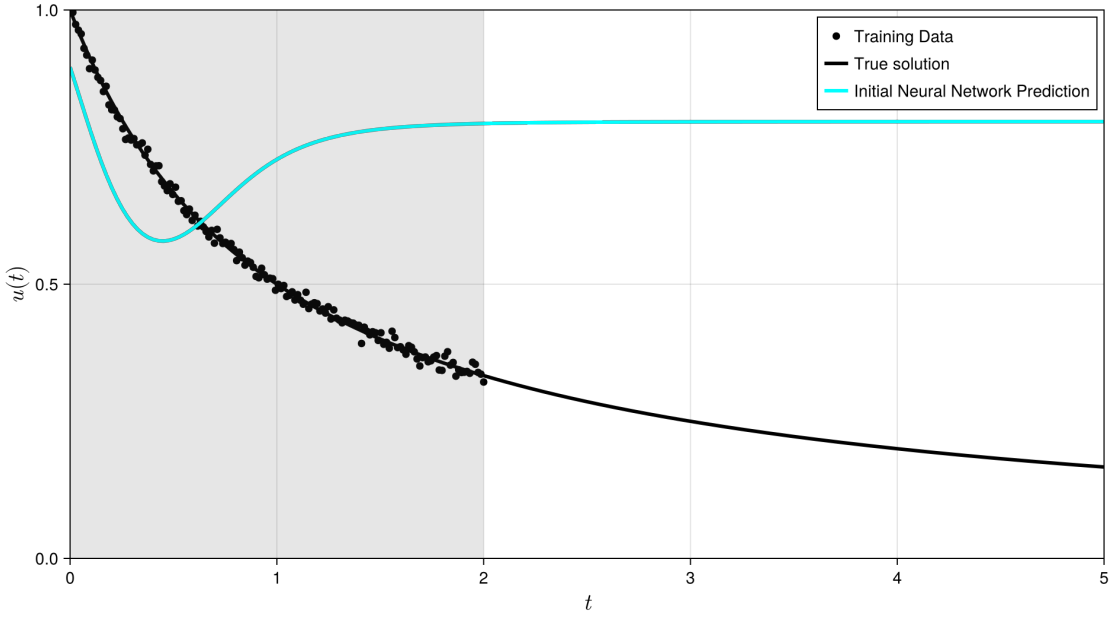
Define Neural Network model, $NN(t;\vec{p})$. One input layer and one output layer with one node. Two hidden layers with 5 nodes and using the tanh activation function for the input and hidden layers.

In [46]:
```
rng = Xoshiro(random_initial_seed)
model = Chain(Dense(1, 5, tanh),
              Dense(5, 5, tanh),
               Dense(5, 1))
pinit, st = Lux.setup(rng, model)
```

```
((layer_1 = (weight = Float32[2.2234793; -0.9252919; … ; 2.651382; 2.54423
55;;], bias = Float32[-0.7991946, -0.81577146, 0.578336, 0.3968519, 0.1644
5601]), layer_2 = (weight = Float32[-0.7375973 1.2798887 … -0.6285871 1.17
32625; 0.39126006 -0.93281496 … -0.17190742 -0.29541454; … ; -1.0276101 0.
02352893 … 0.4088462 0.524053; 0.45034525 0.9984187 … -0.4296156 -0.93543
3], bias = Float32[-0.42017847, 0.115000576, -0.37138525, -0.41014948, 0.0
13214716]), layer_3 = (weight = Float32[-0.5275636 0.21754348 … -0.7123893
-0.3478577], bias = Float32[-0.00041023595])), (layer_1 = NamedTuple(), la
yer_2 = NamedTuple(), layer_3 = NamedTuple()))
```

Plot the initial prediction of the Neural Network with the data

In [47]:
```
u_pred_init,_=model(Array(tsteps2)',pinit, st)

fig = CairoMakie.Figure(size = (960, 540),backgroundcolor = :transparent)
ax = CairoMakie.Axis(fig[1, 1],
    backgroundcolor = :transparent,
    xlabelsize=20,
    ylabelsize=20,
    xlabel=L"t",ylabel=L"u(t)",title="")
CairoMakie.scatter!(ax,tsteps, data[:];color=:black,label="Training Data"
CairoMakie.lines!(ax,tsteps2, data_true[:]; linewidth = 3,color=:black,la
CairoMakie.lines!(ax,tsteps2, u_pred_init[:];linewidth = 3,color=:cyan,la
CairoMakie.vspan!(ax,tspan[1], tspan[2];color=(:grey, 0.1))
axislegend(ax;position=:rt)
xlims!(ax,tspan2[1],tspan2[2])
ylims!(ax,0,1)
#save("figures/01_InitialNNPrediction.png", fig)
fig
```



As can be seen the prediction from the Neural Network is not good. This is because the parameters of the Neural network has not been "trained" yet.

To train the Neural Network we first need to define the loss function.

$$\mathcal{L}(\vec{p}) = \frac{1}{2}\frac{1}{N}\sum_{i=1}^{N}(NN(t_i;\vec{p}) - u_i)^2$$

In [48]:
```
function loss(p,(data,tsteps))
    cost=0.0
    upred,_=model(Array(tsteps)', p, st)
    cost=0.5*sum((upred.-data).^2)/length(tsteps)
    return cost
end
```

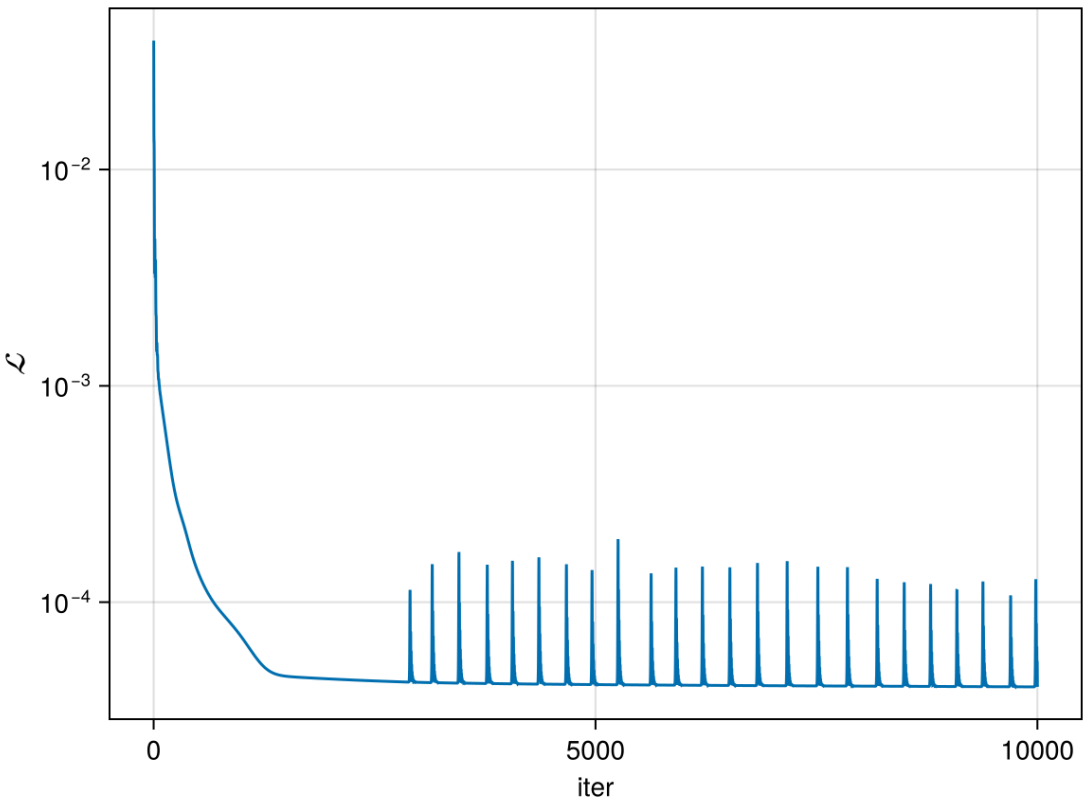
```
loss (generic function with 1 method)
```

In [49]:
```
# Initialize a vector to store loss values
loss_history = Float64[]

callback = function (p, l)
    if p.iter % 1000 == 0
        println("Iteration: $(p.iter), Loss: $l")
    end
    push!(loss_history, l)
    return false
end

adtype = Optimization.AutoZygote()
optf = Optimization.OptimizationFunction((p,x)->loss(p,(data,tsteps)), ad
optprob = Optimization.OptimizationProblem(optf,ComponentArray(pinit),(da
result = Optimization.solve(optprob, OptimizationOptimisers.Adam(learning
```

```
Iteration: 1000, Loss: 6.847937157867677e-5
Iteration: 2000, Loss: 4.415460605176937e-5
Iteration: 3000, Loss: 4.260083123210043e-5
Iteration: 4000, Loss: 4.194925107418241e-5
Iteration: 5000, Loss: 4.2407908454050866e-5
Iteration: 6000, Loss: 4.131801775976939e-5
Iteration: 7000, Loss: 4.1094090570005194e-5
Iteration: 8000, Loss: 4.091217706940318e-5
Iteration: 9000, Loss: 4.074297827510309e-5
Iteration: 10000, Loss: 4.126270113020414e-5
Iteration: 10000, Loss: 4.059346129063645e-5
retcode: Default
u: ComponentVector{Float32}(layer_1 = (weight = Float32[1.6278766; -1.0711
069; … ; 2.4053516; 2.4631689;;], bias = Float32[-0.7043673, -1.1847607,
0.75811803, -0.94932044, 0.3518776]), layer_2 = (weight = Float32[-0.74261
45 1.320534 … -0.8846471 1.1136441; 0.049421556 -0.7915293 … -0.41783613 -
0.4746754; … ; -0.6934737 -0.112823725 … 0.6685686 0.68385386; 0.9948004
1.2691445 … 0.053878766 -0.58314466], bias = Float32[-0.50279677, 0.021658
57, -0.6949699, -0.31890574, -0.36861745]), layer_3 = (weight = Float32[-
0.47502422 0.029382046 … -0.65523386 -0.29399976], bias = Float32[-0.02610
6082]))
```

Plot the loss history

In [50]:
```
fig = CairoMakie.Figure()
ax = CairoMakie.Axis(fig[1, 1],yscale=log10,xlabel="iter",ylabel=L"\mathc
CairoMakie.lines!(ax,loss_history)
fig
```



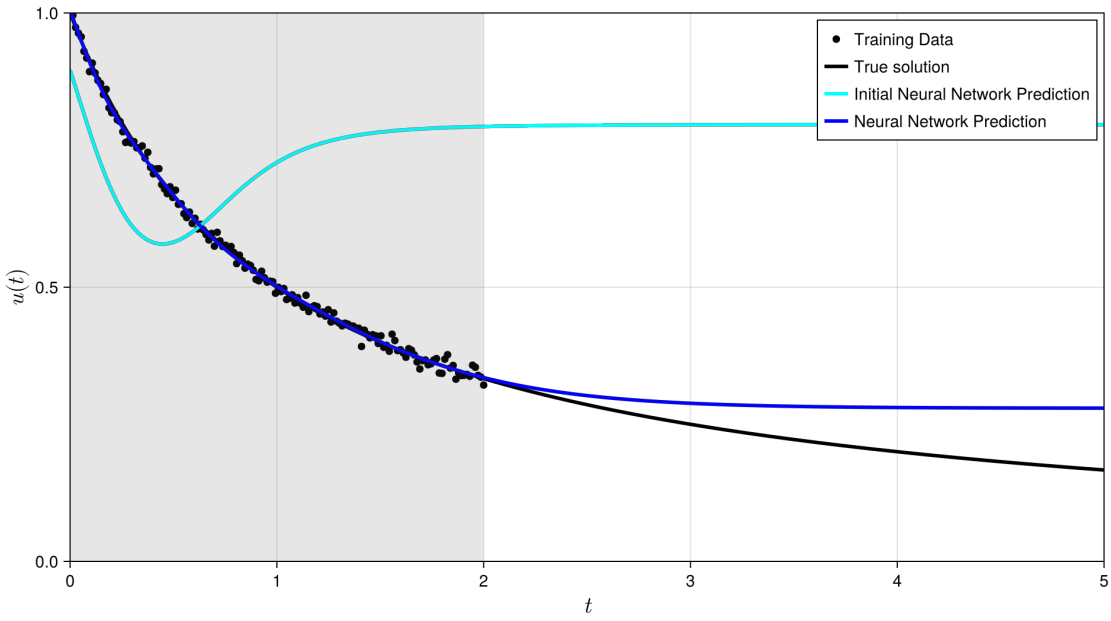
Using the model to predict the solution for $t \in [0, 5]$.

In [51]:
```
u_pred,_=model(Array(tsteps2)',result.u, st)
```

```
(Float32[1.0055041 0.9788297 … 0.27950493 0.27949548], (layer_1 = NamedTup
le(), layer_2 = NamedTuple(), layer_3 = NamedTuple()))
```

Visualise the outcomes of the training

In [52]:
```
fig = CairoMakie.Figure(size = (960, 540),backgroundcolor = :transparent)
ax = CairoMakie.Axis(fig[1, 1],
    backgroundcolor = :transparent,
    xlabelsize=20,
    ylabelsize=20,
    xlabel=L"t",ylabel=L"u(t)",title="")
CairoMakie.scatter!(ax,tsteps, data[:];color=:black,label="Training Data"
CairoMakie.lines!(ax,tsteps2, data_true[:]; linewidth = 3,color=:black,la
CairoMakie.lines!(ax,tsteps2, u_pred_init[:];linewidth = 3,color=:cyan,la
CairoMakie.lines!(ax,tsteps2, u_pred[:];linewidth = 3,color=:blue,label="
CairoMakie.vspan!(ax,tspan[1], tspan[2];color=(:grey, 0.1))
axislegend(ax;position=:rt)
xlims!(ax,tspan2[1],tspan2[2])
ylims!(ax,0,1)
#save("figures/01_NeuralNetworkPrediction.png", fig)
fig
```



As you can see from the plot above the Neural Network model works pretty well in $t \in [0, 2]$, where there is training data. However, in regions where there is no training data, $t \in [2, 5]$, the Neural Network model does not work very well.

Workshop Demo 01: Try changing the training time to $t \in [0, 1]$. What happens to the prediction solution for $t \in [1, 5]$?

Workshop Demo 02: Rerun this example by setting the noise=0.01.
